



**U N I V E R S I D A D
D E L A F R O N T E R A**

ICC708-1
Programación avanzada

**Implementación y Visualización de Algoritmos de
Búsqueda**

Integrantes: Diego Darwittg

Docente: Dr. Ricardo Gacitúa

1. Decisión de diseño

Carga de información: los módulos leerLaberinto.py y leerGrafo.py incluyen la lectura y el análisis de los archivos CSV, retornando estructuras nativas (numpy.ndarray para el laberinto, diccionarios para coordenadas y adyacencia del grafo). Esto separa la entrada de datos de la lógica de presentación y algoritmos.

Visualización e interacción: ventanaPrincipal.py establece la clase VisualizadorLaberinto, que gestiona la interfaz gráfica (matplotlib), el control de eventos y la implementación de los algoritmos.

2. Comparación de algoritmos

Laberinto

(Todas las comparaciones fueron hechas en una matriz de 10x10 donde el punto inicial y el final estan en es)

Algoritmo	Costo	Ruta(pasos)	visitados (Celdas)
BFS	19	20	110
DFS	99	100	100
UCS	19	20	110
A*	19	20	110

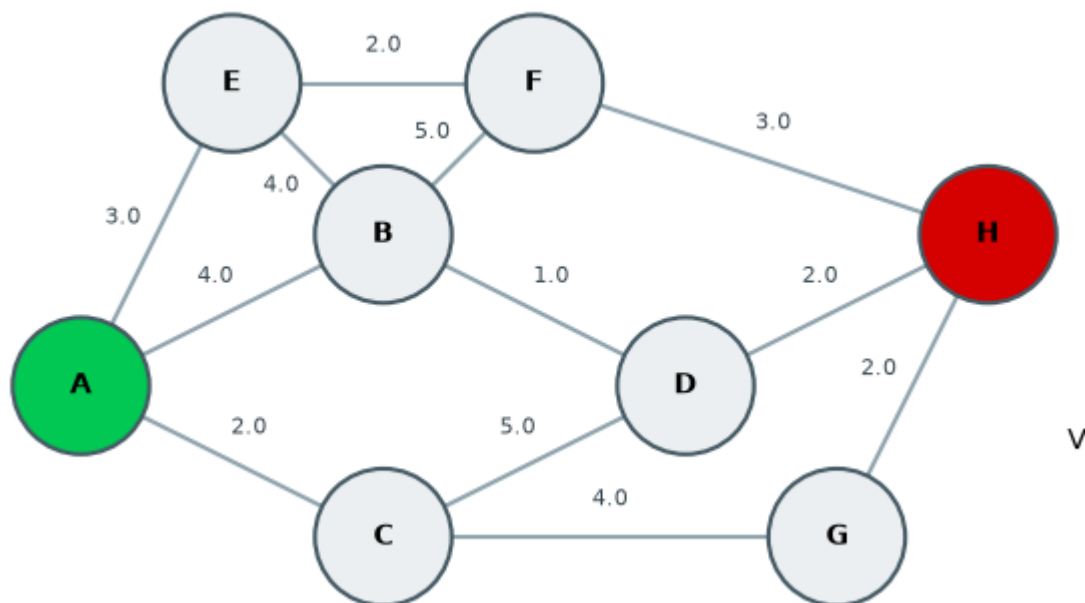
Puntos listos — elige algoritmo y presiona Buscar



Grafos

(Todas las comparaciones fueron hechas en el mismo grafo desde A hasta -> H)

Algoritmo	COsto	Ruta	Visitados
BFS	7.0	A->B->D->H	8
DFS	15	A->E->B->F->H	5
UCS	7	A->B->D->H	8
A*	7	A->B->D->H	5



Laberinto (matriz 10x10)

BFS, UCS y A* obtuvieron exactamente el mismo resultado: costo 19, 20 pasos y 110 celdas visitadas. Esto se explica porque en un laberinto con costo uniforme por paso, los tres algoritmos son equivalentes en cuanto al camino encontrado. La diferencia de A* respecto a los otros es que en teoría debería visitar menos nodos gracias a la heurística, pero en este caso particular el laberinto no le dio ventaja suficiente para reducir las celdas exploradas.

DFS en cambio encontró un camino de 100 pasos con costo 99, lo que demuestra que no busca el óptimo sino que sigue una rama hasta el final. Curiosamente visitó menos celdas (100), porque se fue directo por un camino largo sin explorar el espacio lateralmente.

Grafos (A → H)

A* encontró el camino óptimo (costo 7, ruta A→B→D→H) visitando solo 5 nodos, igualando a DFS en nodos visitados pero superando a los demás con su solución. DFS llegó a H por un camino de costo 15 (A→E→B→F→H), más del doble del óptimo.

BFS y UCS encontraron el camino óptimo pero revisaron 8 nodos, ya que exploran de forma más exhaustiva sin guía.

Conclusiones.

- DFS es el algoritmo menos adecuado cuando se busca el camino de menor costo, tanto en laberintos como en grafos ponderados.
- BFS es óptimo en entornos de costo uniforme pero no escala bien en grafos con pesos variables.
- UCS garantiza la solución óptima en cualquier grafo ponderado, a costa de explorar más nodos.
- A* es el más eficiente cuando se dispone de una buena heurística, logrando el óptimo con menos nodos visitados. Su ventaja se hace más evidente en grafos que en laberintos simples.

Nota extra: Profesor para las pruebas yo ejecute el programa en mi terminal de linux por facilidad, porfavor si surge algún error comunicarlo y intentare solucionarlo.